CHAPTER 1

INTRODUCTION

Many of the studies in recent years admit that humans reply and react to music and this music has a high impression on the activity of the human brain. In one examination of the explanations why people hear music, researchers discovered that mu-sic played a crucial role in relating arousal and mood. Two of the most important functions of music are it is ability is participants rated to help them achieve a good mood and become more self-aware. Musical preferences have been demonstrated to be highly related to personality traits and moods.

The meter, timbre, rhythm, and pitch of music are managed in areas of the brain that affects emotions and mood. Interaction between individuals may be a major aspect of lifestyle. It reveals perfect details and much of data among humans, whether they are in the form of body language, speech, facial expression, or emotions. Nowadays, emotion detection is considered the most important technique used in many applications such as smart card applications, surveillance, image database investigation, criminal, video indexing, civilian applications, security, and adaptive \human-computer interface with multimedia environments. With the increase in technology for digital signal processing and other effective feature extraction algorithms, automated emotion detection in multimedia attributes like music or movies is growing rapidly and this system can play an important role in many potential applications like human-computer interaction systems and music entertainment. We use facial expressions to propose a recommender system for emotion recognition that can detect user emotions and suggest a list of appropriate songs. The proposed system detects the emotions of a person, if the person has a negative emotion, then a certain playlist will be shown that includes the most related types of music that will enhance his mood. And if the emotion is positive, a specific playlist will be presented which contains different types of music that will inflate the positive emotions.

1.1 Motivation

- Personalize music for users' emotional states
- Create emotional connections through music

- Enhance moods with tailored recommendations
- Facilitate music discovery
- Enrich user experience
- Innovate with AI-driven recommendations
- Scale with Spotify's vast music library

1.2 Scope

Develop a Python application that utilizes natural language processing (NLP) to analyze user input describing their emotional state, and generates personalized music recommendations based on their emotions. By integrating with the Spotify API, the application maps user emotions to corresponding music genres, playlists, or tracks, and retrieves matching music recommendations. This project aims to create a unique and engaging user experience by combining AI-driven emotional analysis with music information retrieval, providing users with a curated list of music tailored to their emotional needs.

1.3 Objectives

- Emotions-based music recommendations using the Spotify API and Python can enhance the listening experience by tailoring it to a user's emotional state.
- By analyzing a user's emotional input, such as happiness, sadness, or relaxation, the program can generate a personalized playlist that resonates with their current mood.
- This is achieved by leveraging the Spotify API's vast music library and Python's natural language processing capabilities to identify tracks with corresponding emotional characteristics, such as tempo, genre, and lyrics.
- The resulting playlist can help users process, validate, or shift their emotions, creating a more immersive and empathetic connection with the music.
- Additionally, this approach can expand the user's musical horizons by introducing them to new artists and genres that align with their emotional preferences.

1.4 Features

- Real time expression detection and song recommendations.
- Playlists fetched from Spotify using API.

- Neomorphism UI for website.

1.5 Summary

Emotions-based music recommendations using the Spotify API with Python is a project that detects user emotions and creates personalized playlists to match their mood. By leveraging natural language processing, machine learning, and audio feature extraction, the system provides tailored music suggestions. Integration with the Spotify API enables access to track metadata and user data, while a real-time system with a user-friendly interface ensures a seamless experience. Through continuous improvement based on user feedback, this project combines AI, music, and emotion to provide a unique and engaging user experience.

CHAPTER 2

TECHNOLOGY DETAILS

2.1 Keras

Keras is a high-level neural networks API that provides an easy-to-use interface for building and training deep learning models, written in Python and capable of running on top of several backend engines, including TensorFlow, Theano, and Microsoft Cognitive Toolkit (CNTK). Its primary goal is to enable fast experimentation with deep neural networks, focusing on being user-friendly, modular, and extensible. Keras allows for easy and fast prototyping, supporting both convolutional and recurrent neural networks, making it an ideal tool for beginners and experts alike. It offers an intuitive API for building and training neural networks, with a focus on simplicity and ease of use, while still allowing for customization and extension.

Keras is designed to be modular, allowing users to easily build and combine different neural network components, such as layers, models, and optimizers. This modularity makes it easy to experiment with different neural network architectures and techniques, and to share and reuse code. Additionally, Keras can be easily extended to support new neural network layers, models, and optimizers, making it a flexible tool for research and development. With Keras, developers can quickly build and test neural network models, and it supports GPU acceleration, making it possible to train large neural networks quickly and efficiently.

Keras has a wide range of applications, including image classification, speech recognition, natural language processing, and time series prediction. It can be used to build convolutional neural networks (CNNs) for image classification tasks, such as recognizing objects in images, and recurrent neural networks (RNNs) for speech recognition tasks, such as transcribing spoken words into text. Keras can also be used to build RNNs and long short-term memory (LSTM) networks for natural language processing tasks, such as language translation and text generation, and for time series prediction tasks, such as predicting stock prices or weather patterns.

2.2 Tensorflow

TensorFlow is an open-source software library for numerical computation, particularly well-suited and fine-tuned for large-scale Machine Learning (ML) and Deep Learning (DL) tasks. Its primary use is in developing and training artificial neural networks, particularly deep neural networks. TensorFlow allows developers to easily implement popular DL architectures such as Convolutional Neural Networks (CNNs), Recurrent Neural Networks (RNNs), and Autoencoders.

TensorFlow's architecture is designed to facilitate research, development, and production of ML models. It provides an ecosystem of tools, libraries, and community resources that enables developers to push the boundaries of ML and DL. TensorFlow's core features include automatic differentiation, gradient descent optimization, and support for GPU acceleration, making it an ideal choice for large-scale ML tasks. Additionally, TensorFlow provides tools for visualizing and understanding ML models, including TensorBoard, a web-based interface for exploring and debugging models.

2.3 Spotipy

Spotipy is a lightweight Python library that provides a simple and intuitive interface to the Spotify Web API. It allows developers to easily access and manipulate Spotify data, such as tracks, albums, artists, playlists, and user information. With Spotipy, you can build a wide range of applications, including music recommendation systems, playlist generators, and music data analytics tools.

Spotipy's core features include support for all Spotify API endpoints, including search, lookup, browse, and user libraries. It also provides functionality for user authentication, allowing developers to access user-specific data and perform actions on behalf of the user. Spotipy's API is designed to be easy to use, with simple and consistent method names and parameters.

Spotipy has a wide range of applications, including music streaming services, data analysis and visualization, and music recommendation systems. It can be used to build applications that provide personalized music recommendations, generate playlists based on user preferences, or analyze music trends and popularity. Spotipy's simplicity and flexibility make it a popular choice among developers and data scientists working with music data.

2.4 Flask

Flask is a Python web framework that provides a lightweight and flexible way to build web applications. It's ideal for small to medium-sized projects, prototyping, and proof-of-concepts, where a simple and easy-to-use framework is preferred. Flask's minimalist approach allows developers to build web applications quickly, without the overhead of a larger framework.

Flask's key features include a modular design, support for routing, templating, and internationalization, as well as a built-in development server. Its API is designed to be simple and intuitive, making it easy for developers to get started quickly. Flask also has a rich set of extensions and libraries available, which can be easily installed to add additional functionality.

Flask is well-suited for building web services, RESTful APIs, and web applications that require a high degree of customization. Its flexibility and extensibility make it a popular choice among developers who want to build web applications quickly and with minimal overhead. Flask is also a great choice for building data-driven applications, such as data visualization tools, dashboards, and analytics platforms.

Flask's community is large and active, with a wealth of documentation, tutorials, and resources available. The framework is well-maintained and regularly updated to support new features and best practices. Flask is compatible with Python 3.6+ and can be easily installed using pip. With Flask, developers can build web applications and APIs quickly and easily, and its flexibility makes it an ideal choice for a wide range of web development tasks

CHAPTER 3

SOFTWARE AND HARDWARE REQUIREMENTS

This section delineates the types of information and materials essential for initiating the project implementation phase.

3.1 HARDWARE REQUIREMENTS

Any personal computer that meets the following specifications:

1) Processor- core i3 and above.

2) Speed- 1.2 GHZ.

3) Ram- 4 GB.

4) Hard disk- 20 GB

3.2. SOFTWARE REQUIREMENTS

Any personal computer that will meet these following Specifications will work:

1) Operating System - Linux, Windows 7 and above.

2) Language – Python

3) Tools- Visual Studio Code.

CHAPTER 4

SYSTEM DESIGN

This project is of a deep learning model architecture, specifically a convolutional neural network (CNN) designed for image classification, likely for emotion recognition from images.

4.1 Model Architecture

1. **Sequential Model:** The model is built layer by layer in a linear sequence, meaning each layer is added one after the other, with the output of one layer becoming the input for the next.
2. **Conv2D Layers:**
    - These are convolutional layers, which extract features from images using filters (also known as kernels).
    - The different filter sizes (32, 64, 128) allow the model to detect various levels of detail in the images.
    - The 'relu' (Rectified Linear Unit) activation function introduces non-linearity to the model, enabling it to learn complex patterns.
3. **Maxpool2d Layers:**
    - These are pooling layers, which downsample the feature maps, reducing spatial dimensions while retaining important information.
    - The pool size (2,2) means they take the maximum value from each 2x2 patch, effectively reducing the image size and highlighting prominent features.
4. **Dropout Layers:**
    - These layers randomly drop out 25% of neurons during training, preventing overfitting (when the model becomes too specialized to the training data).
    - Dropout helps improve generalization, allowing the model to perform better on unseen data.
5. **Dense Layers:**
    - These are fully connected layers, used for classification

- The final Dense layer likely has a softmax activation function (not mentioned explicitly), which outputs a probability distribution over the possible emotion classes.

4.2 Hyperparameters and Training

1. Loss Function: Categorical Crossentropy :

- Used for multi-class classification problems where each sample belongs to one of multiple classes (e.g., dog, cat, bird, etc.).
- Measures the difference between the model's predicted probabilities and the true labels.
- Formula: -∑(true_label * log(predicted_probability))
- Goal: Minimize the loss by adjusting model weights to predict accurate probabilities.

2. Optimizer: Adam :

- A popular stochastic gradient descent (SGD) optimizer.
- Updates model weights based on the gradient of the loss function with respect to each weight.
- Uses adaptive learning rates for each parameter, making it more efficient and effective.
- Combines the benefits of two other optimizers: AdaGrad and RMSProp.

- Parameters:

- Learning rate (α): controls how quickly weights are updated.
- Beta1 (β1) and Beta2 (β2): control the decay rates of the moving averages.

3. Metric: Accuracy

- Evaluates the model's performance by measuring the proportion of correctly classified samples.
- Formula: (TP + TN) / (TP + TN + FP + FN)

  Where:

  TP: True Positives (correctly classified positive samples)

  TN: True Negatives (correctly classified negative samples)

FP: False Positives (misclassified positive samples)

FN: False Negatives (misclassified negative samples)

- Goal:

- Maximize accuracy by adjusting model weights to make correct predictions.

In summary, these hyperparameters and training components work together to train a model that accurately classifies samples into multiple classes. The loss function measures the difference between predictions and true labels, the optimizer updates weights to minimize the loss, and the accuracy metric evaluates the model's performance.

CHAPTER 5

IMPLEMENTATION

5.1 System Overview

Music recommendation based on facial emotion recognition contains four modules.

- **Real-Time Capture:** In this module, the system is to capture the face of the user correctly.
- **Face Recognition:** Here it will take the user's face as input. The convolutional neural network is programmed to evaluate the features of the user image.
- **Emotion Detection:** In this section extraction of the features of the user image is done to detect the emotion and depending on the user's emotions, the system will generate captions.
- **Music Recommendation:** Song is suggested by the recommendation module to the user by mapping their emotions to the mood type of the song.

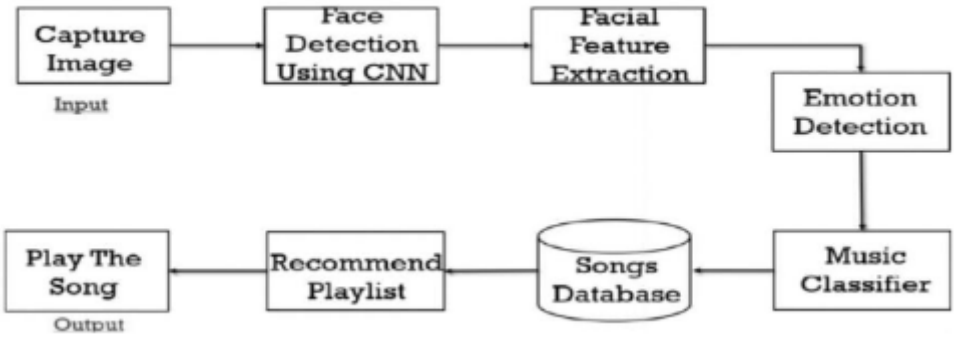

***Figure 1**. Block diagram of the* proposed system.

5.2 Database Description

We built the Convolutional Neural Network model using the Kaggle dataset. The database is FER2013 which is split into two parts training and testing dataset. The training dataset consists of 24176 and the testing dataset contains 6043 im-ages. There are 48x48 pixel grayscale images of faces in the dataset. Each image in FER-2013 is labeled as one of

five emotions: happy, sad, angry, surprise, and neutral. The faces are automatically registered so that they are more or less centered in each image and take up about the same amount of space. The images in FER-2013 contain both posed and unposed headshots, which are in grayscale and 48x48 pixels. The FER-2013 dataset was created by gathering the results of a Google image search of every emotion and synonyms of the emotions. FER systems being trained on an imbalanced dataset may perform well on dominant emotions such as happy, sad, angry, neutral, and surprised but they perform poorly on the under -represented ones like disgust and fear. Usually, the weighted-SoftMax loss approach is used to handle this problem by weighting the loss term for each emotion class supported by its relative proportion within the training set. However, this weighted-loss approach is predicated on the SoftMax loss function, which is reported to easily force features of various classes to stay apart without listening to intra-class compactness. One effective strategy to deal with the matter of SoftMax loss is to use an auxiliary loss to coach the neural network. To treating missing and Outlier values we have used a loss function named categorical crossentropy. For each iteration, a selected loss function is employed to gauge the error value. So, to treating missing and Outlier values, we have used a loss function named categorical crossentropy.

***Figure 2**. Samples from FER2013 dataset*

5.3 Emotion Detection Module

5.3.1 Face Detection

Face detection is one of the applications which is considered under computer vision technology. This is the process in which algorithms are developed and trained to properly locate faces or objects in object detection or related system in images. This detection can be real-time from a video frame or images. Face detection uses such classifiers, which are algorithms that detect what's either a face (1) or not a face (0) in an image. Classifiers are trained to detect faces using numbers of images to get more accuracy. OpenCV uses two sorts of classifiers, LBP (Local Binary Pattern) and Haar Cas-cades. A Haar classifier is used for face detection where the classifier is trained with pre -defined varying face data which enables it to detect different faces accurately. The main aim of face detection is to spot the face within the frame by reducing external noises and other factors. It is a machine learning-based approach where the cascade function is trained with a group of input files. It is supported the Haar Wavelet technique to research pixels inside the image into squares by function [9]. This uses machine learning techniques to urge a high degree of accuracy from what's called "training data".

5.3.2 Feature Extraction

While performing feature extraction, we treat the pre-trained network that is a sequential model as an arbitrary feature ex-tractor. Allowing the input image to pass on it forward, stopping at the pre-specified layer, and taking the outputs of that layer as our features. Starting layers of a convolutional network extract high-level features from the taken image, so use only a few filters. As we make further deeper layers, we increase the number of the filters to twice or thrice the dimension of the filter of the previous layer. Filters of the deeper layers gain more features but are computationally very intensive.

Doing this we utilized the robust, discriminative features learned by the Convolution neural network [10]. The outputs of the model are going to be feature maps, which are an intermediate representation for all layers after the very first layer. Load the input image for which we want to view the Feature map to know which features were prominent to classify the image. Feature maps are obtained by applying Filters or Feature detectors to the input image or the feature map output of the prior layers. Feature map visualization will provide insight

into the interior representations for specific i n-put for each of the Convolutional layers within the model.

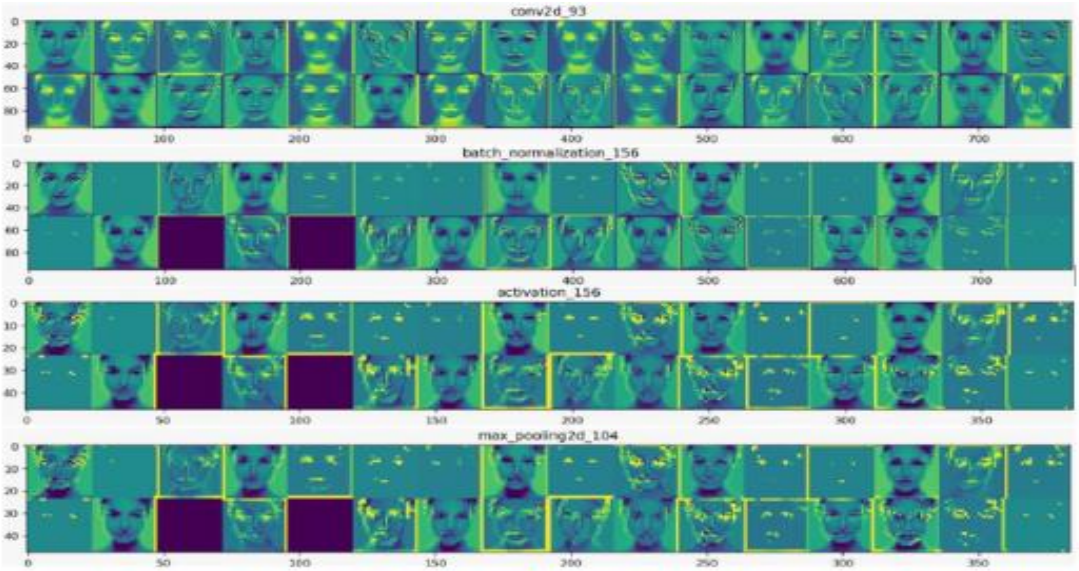

Figure 3. *Visualization of The Feature Map.*

5.3.3 Emotion Detection

Convolution neural network architecture applies filters or feature detectors to the input image to get the feature maps or activation maps using the Relu activation function [11]. Feature detectors or filters help in identifying various features pre-sent in the image such as edges, vertical lines, horizontal lines, bends, etc. After that pooling is applied over the feature maps for invariance to translation. Pooling is predicted on the concept that once we change the input by a touch amount, the pooled outputs don't change. We can use any of the pooling from min, average, or max. But max-pooling provides better performance than min or average pooling. Flatten all the input and giving these flattened inputs to a deep neural network which are outputs to the class of the object.

The class of the image will be binary, or it will be a multi-class classification for identifying digits or separating various apparel items. Neural networks are as a black box, and learned features in a Neural Network are not interpretable. So basically, we give an input image then the CNN model returns the results [10]. Emotion detection is performed by loading the model which is trained by weights using CNN. When we take the real-time image by a user

then that image was sent to the pre-trained CNN model, then predict the emotion and adds the label to the image.

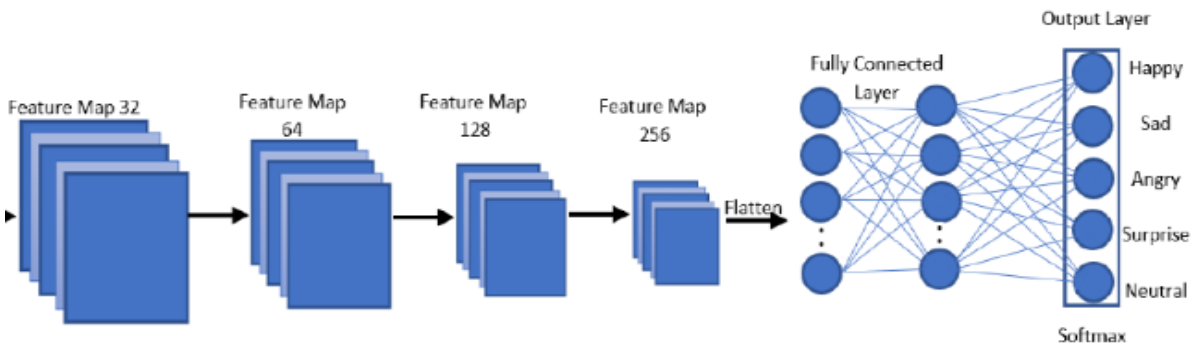

Figure 4. *Convolution neural network Architecture.*

5.4 Music Recommendation Module

5.4.1 Songs Database

We created a database for Bollywood Hindi songs. It consists of 100 to 150 songs per emotion. As we all know music is undoubtedly involved in enhancing our mood. So, suppose a user is sad then the system will recommend such a music playlist which motivates him or her and by this automatic mood will be delighted.

5.4.2 Music Playlist Recommendation

By using the emotion module real-time emotion of the user is detected. This will give the labels like Happy, Sad, Angry, Surprise, and Neutral. Using the os.listdir() method in python we connected these labels with the folders of the songs database which we have created. Table 1 shows the list of songs. This method of os.listdir() is used to get the list of any file in the specified directories. Example for Happy :

```
if label== 'Happy':

os.chdir("C:/Users/User/Downloads/Happy")

self.mood.set("You are looking happy, I am playing song for You")

# Fetching Songs songtracks = os.listdir()
```

CHAPTER 6

RESULTS AND SNAPSHOTS

6.1 Song Recommendations for Sad and Happy Faces

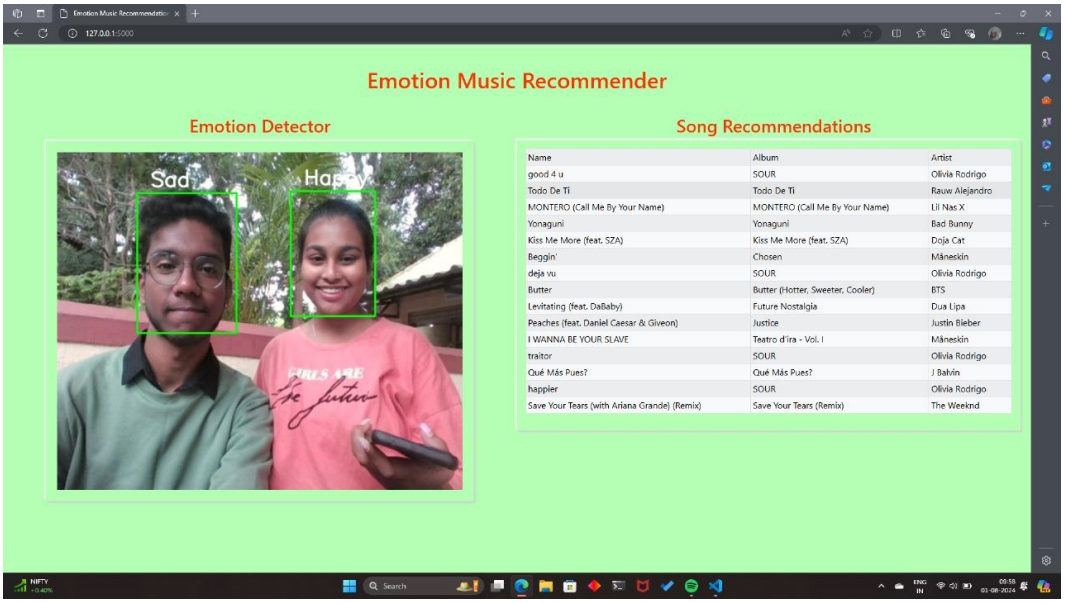

Figure 5. *Detects Happy and Sad face and recommend songs*

6.2 Song Recommendations for Suprised Face

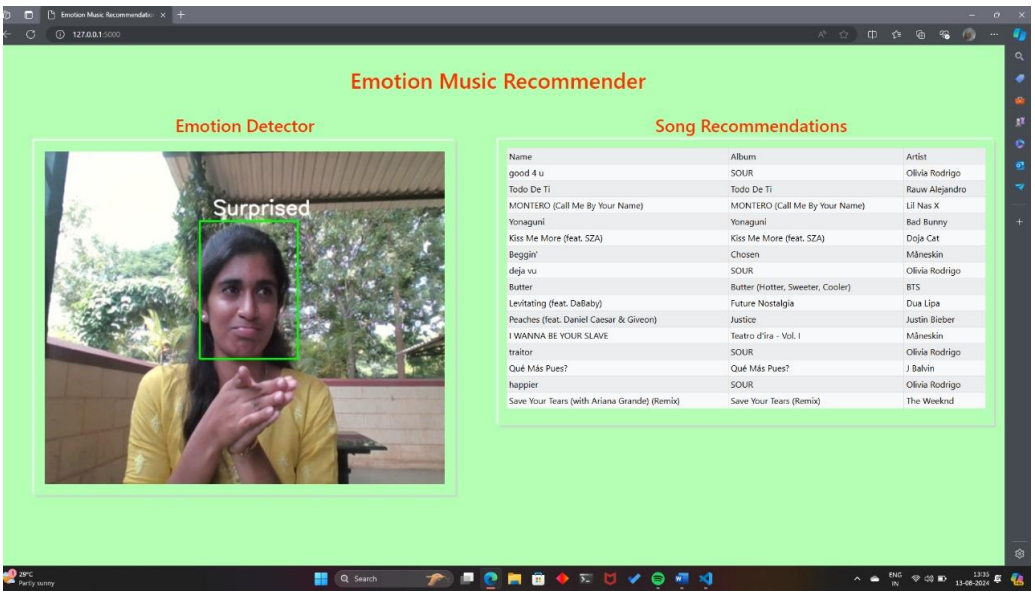

Figure 6. *Detects Surprise face and recommend songs*

6.3 Song Recommendations for Neutral Face

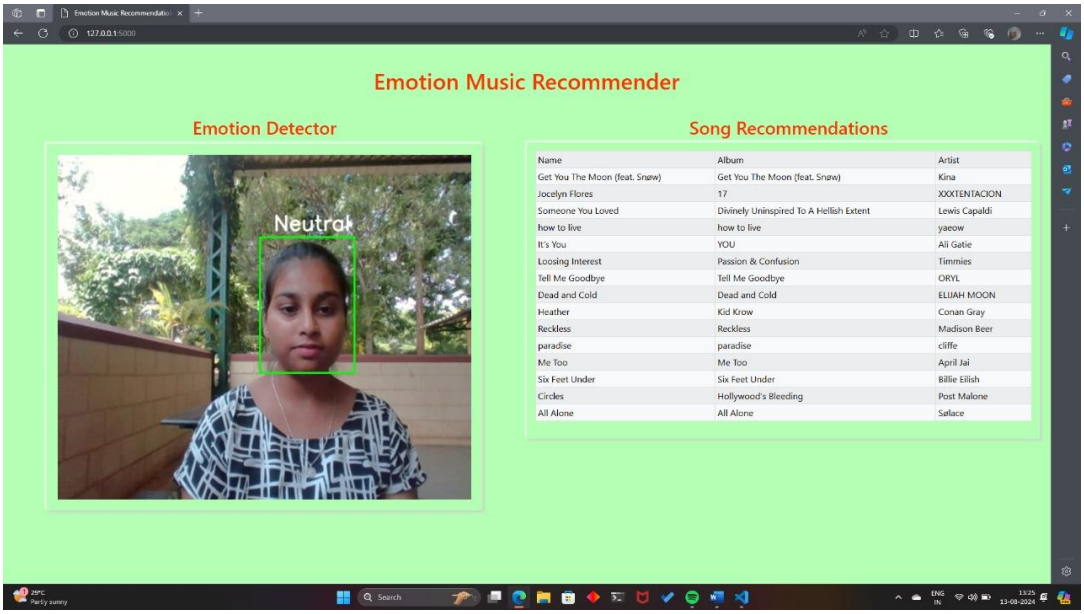

Figure 7. *Detects Neutral face and recommend songs*

6.3 Song Recommendations for Fearful Face

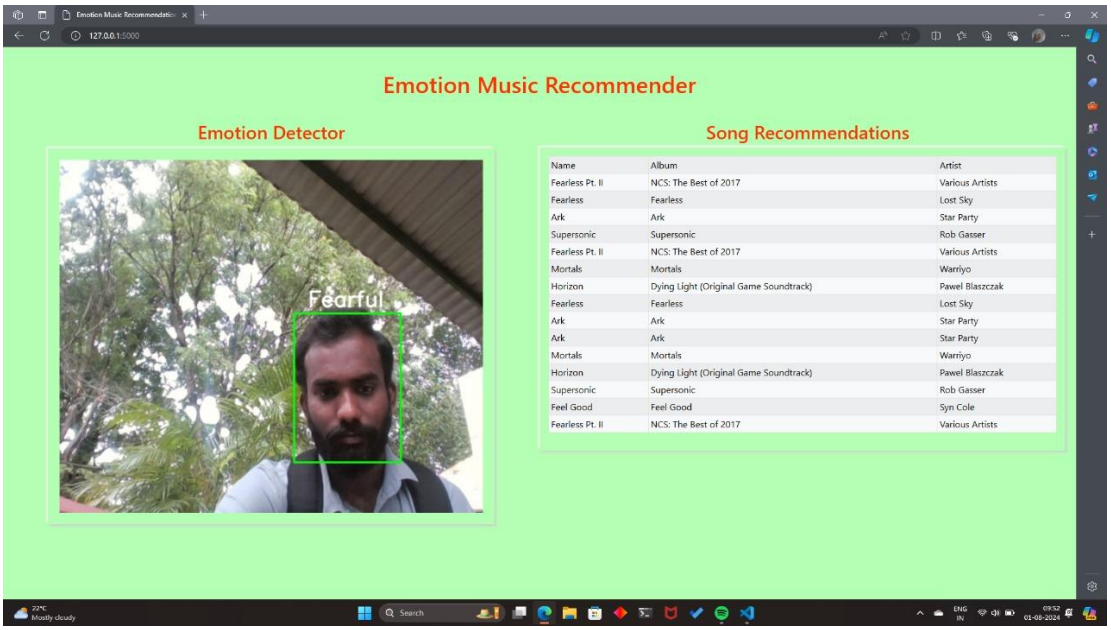

Figure 8. *Detects Fearful face and recommend songs*

CHAPTER 7

CONCLUSION AND FUTURE ENHANCEMENT

In this system, we provide an overview of how music can affect the user's mood and how to choose the right music tracks to improve the user's moods. The implemented system can detect the user's emotions. The emotions that the system can detect were happy, sad, angry, neutral, or surprised. After determining the user's emotion, the proposed system provided the user with a playlist that contains music matches that detected the mood.

This system, although completely functioning, does have scope for improvement in the future. There are various aspects of the application that can be modified to produce better results and a smoother overall experience for the user. Some of these that an alternative method, based on additional emotions which are excluded in our system as disgust and fear.

The future scope within the system would style a mechanism that might be helpful in music therapy treatment and help the music therapist to treat the patients suffering from mental stress, anxiety, acute depression, and trauma.

BIBLIOGRAPHY

[1] Ramya Ramanathan, Radha Kumaran, Ram Rohan R, Rajat Gupta, and Vishalakshi Prabhu, an intelligent music player based on emo-tion recognition, 2nd IEEE International Conference on Computational Systems and Information Technology for Sustainable Solutions 2017. https://doi.org/10.1109/CSITSS.2017.8447743

[2] Shlok Gilda, Husain Zafar, Chintan Soni, Kshitija Waghurdekar, Smart music player integrating facial emotion recognition and music mood recommendation, Department of Computer Engineering, Pune Institute of Computer Technology, Pune, India, (IEEE),2017. https://doi.org/10.1109/WiSPNET.2017.8299738

[3] Deger Ayata, Yusuf Yaslan, and Mustafa E. Kamasak, Emotion-based music recommendation system using wearable physiological sensors, IEEE transactions on consumer electronics, vol. 14, no. 8, May 2018.https://doi.org/10.1109/TCE.2018.2844736

[4] Ahlam Alrihail, Alaa Alsaedi, Kholood Albalawi, Liyakathunisa Syed, Music recommender system for users based on emotion detection through facial features, Department of Computer Science Taibah University, (DeSE), 2019. https://doi.org/10.1109/DeSE.2019.00188

[5] Research Prediction Competition, Challenges in representation learning: facial expression recognition challenges, Learn facial expres-sion from an image, (KAGGLE).

[6] Preema J.S, Rajashree, Sahana M, Savitri H, Review on facial expression-based music player, International Journal of Engineering Re-search & Technology (IJERT), ISSN-2278-0181, Volume 6, Issue 15, 2018.

[7] AYUSH Guidel, Birat Sapkota, Krishna Sapkota, Music recommendation by facial analysis, February 17, 2020.

[8] CH. sadhvika, Gutta.Abigna, P. Srinivas reddy, Emotion-based music recommendation system, Sreenidhi Institute of Science and Technology, Yamnampet, Hyderabad; International Journal of Emerging Technologies and Innovative Research (JETIR) Volume 7, Is-sue 4, April 2020.